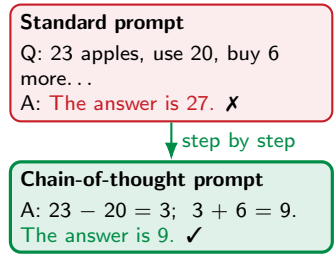
Make the model show its work

Ask a large model a math word problem and demand a bare answer – it often guesses wrong. Ask the *same* model to **think step by step** first, and the answer flips to correct.

No new weights, no finetuning: the ability was latent, and a few words of prompt **elicited** it. But there is a catch – it only works once the model is **big enough**.

And a second catch: a model can reach the right answer through *wrong* steps. If we only reward the final answer, we reward **luck**.

Today: *elicit* the reasoning (Wei 2022), then *reward the steps* (Lightman et al. 2023).



(Wei et al. Fig. 1, the canonical example.)

Outline

Chain-of-thought prompting

Show the model a few worked examples – and it starts working things out.

What chain-of-thought prompting is

A **chain of thought** is a series of intermediate natural-language reasoning steps that lead to the final answer. Two ways to elicit it, no finetuning:

- **Few-shot:** put a handful of exemplars in the prompt as triples ⟨input, *chain of thought*, output⟩ – eight, hand-written, no prompt engineering.
- **Zero-shot:** just append *"Let's think step by step"* (Kojima et al. 2022).

The model is off-the-shelf and **frozen**. CoT changes only the *prompt*, so a single checkpoint gains reasoning on many tasks at once – no per-task training set.

Why it should help: it lets the model *decompose* a hard problem, *spend more compute* on harder inputs, and leaves an *interpretable* trace you can debug.

The one idea: turn one hard leap into easy steps

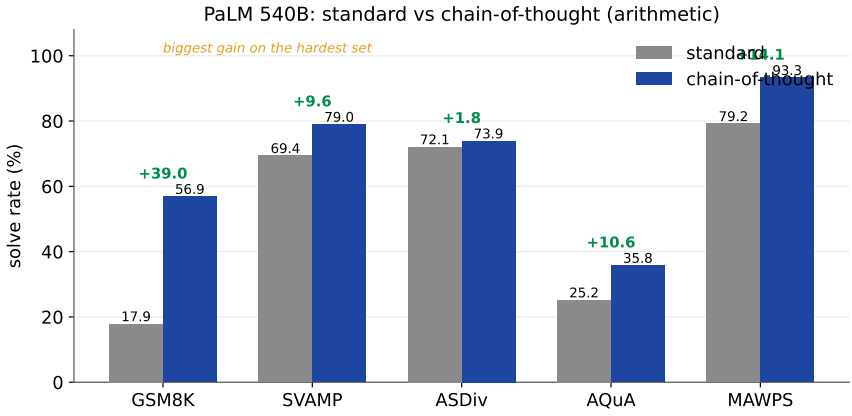
A language model is very good at the **next** token, and much worse at leaping straight from a hard question to its answer in a single shot.

- CoT breaks that one hard leap into a **staircase** of small steps – each an *easy* next-token prediction, conditioned on the steps before it.
- The model walks down to the answer instead of jumping, so each move is one it can actually get right.
- And it must reason **before** committing to an answer – no rationalizing after the fact.

It is *decomposition*: 23 apples $\rightarrow$ $(23 - 20 = 3) \rightarrow (3{+}6 = 9)$. Each line is trivial; the leap from the question straight to "9" is not.

Careful: it is the *reasoning content* that helps, not merely writing more – the ablations a few slides on show extra tokens alone do nothing.

Big gains on arithmetic, commonsense, symbolic



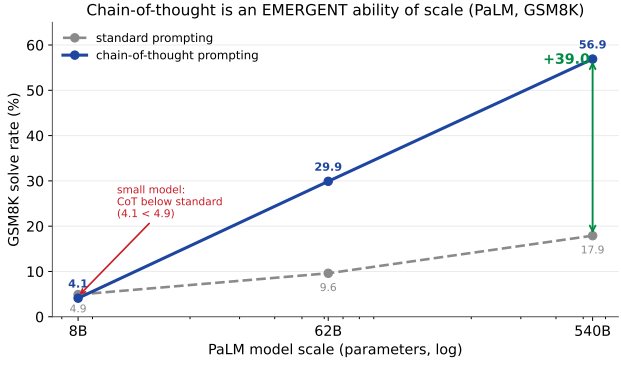
On **PaLM 540B**, CoT lifts GSM8K 17.9→56.9 (+39.0) and sets a new state of the art. Gains are largest on the *hardest* multi-step sets and small on easy one-step ones (ASDiv +1.8). It also carries to commonsense (StrategyQA 69.4→75.6) and symbolic tasks.

Predict first: does CoT help a *small* model?

A 4-year-old learns to "show their work" and improves. Give the same step-by-step prompt to a **small** language model – better, same, or worse?

Predict first: does CoT help a *small* model?

A 4-year-old learns to "show their work" and improves. Give the same step-by-step prompt to a **small** language model – better, same, or worse?



At **8B**, CoT is *worse* than a bare answer ($4.1 < 4.9$): small models write fluent but illogical chains. The benefit **emerges** with scale – it appears only past $\sim$100B parameters.

Why it works: it is the reasoning, not a side effect

Ablations rule out the "easy" explanations (LaMDA 137B, PaLM 540B):

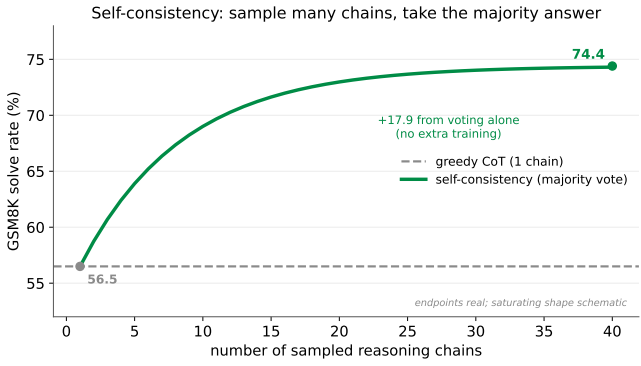
- **Equation only** (emit just the formula, no words): little help on GSM8K – the language matters.
- **Variable compute only** (emit "..." dots of equal length): no gain – extra tokens alone are not it.
- **Reasoning after the answer:** no gain – the model must reason *before* committing, not rationalize afterwards.

The sequential, *natural-language* reasoning is what carries the gain – robust across annotators, exemplars, and model families.

Self-consistency

One chain can go wrong. Sample many, and let them vote.

Sample many chains, take the majority answer



Instead of one greedy chain, **sample** many diverse chains at temperature and **marginalize** over the reasoning: keep the *most frequent final answer* (Wang et al. 2022).

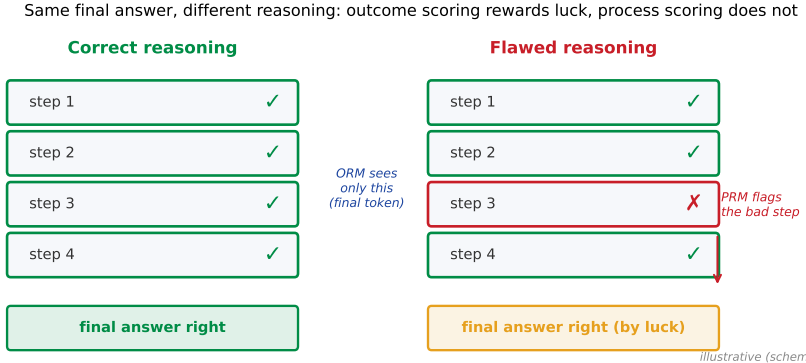
Different correct paths tend to agree on the answer; wrong paths scatter. On PaLM 540B GSM8K this alone lifts 56.5 $\rightarrow$ **74.4**.

No training, no verifier – just more samples at inference. Gains saturate after a few dozen chains.

The reliability gap

A right answer is not a right *reasoning*.

Right answer, wrong steps



A model can reach the correct final answer through a flawed chain. An **outcome** signal (grade the final answer) marks both solutions "correct" – and rewards the lucky-but-wrong one. Even a single logical slip can derail a long solution, so this matters more as problems get harder.

In Wei's own audit, some correct-answer chains were logically wrong – and among wrong answers, $\sim$54% had major reasoning errors.

Let's Verify Step by Step

If wrong steps are the problem, score the steps (Lightman et al. 2023).

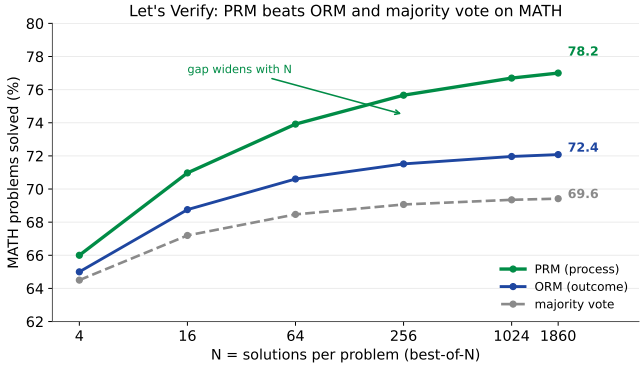
Outcome vs process reward models

Two ways to train a reward model that ranks solutions:

	ORM (outcome)	**PRM** (process)
Feedback on	the *final answer* only	*every reasoning step*
Signal	1 label per solution	1 label per step (pos/neg/neutral)
Supervision	auto (check final answer)	humans label each step
Credit assign.	must guess *where* it failed	knows exactly where

Both score solutions for **best-of-N** search: sample N chains from a fixed generator, keep the one the reward model ranks highest, grade its answer.

The result: PRM wins on MATH



Best-of-1860 on a MATH test subset:

- **PRM 78.2%**
- ORM 72.4%
- majority vote 69.6%

The PRM leads at every N, and the **gap widens** as N grows – it is a better filter over many candidate solutions.

Same generator, same test set – only the *reward signal* differs.

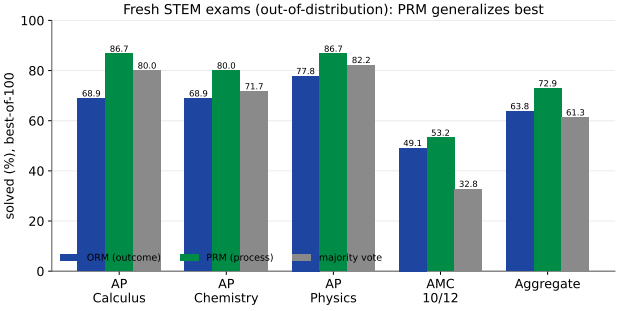
PRM800K: the human step-labels

To train the PRM, humans labelled each step of model-generated MATH solutions:

- **800K** step-level labels over **75K** solutions to **12K** problems – released as **PRM800K**.
- Each step: *positive* / *negative* / *neutral*; supervise only *up to the first mistake* (keeps ORM/PRM comparable).
- **Active learning:** surface "convincing wrong-answer" solutions (rated high by the current PRM but actually wrong) – **2.6**$\times$ more data-efficient than uniform labelling.

Process labels are expensive – there is no auto-grader for a *step*. The active-learning trick is what makes the human budget go far.

More performant *and* more aligned



On *fresh* STEM exams (AP, AMC – released after pretraining), the PRM still wins: **72.9%** aggregate vs ORM 63.8%, majority 61.3%.

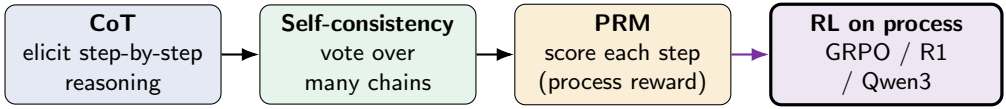
And process supervision is **more aligned**:

- rewards a human-endorsed chain, not just the outcome;
- errors are *located* $\Rightarrow$ interpretable;
- here it is *more* capable too – a **negative alignment tax**.

The through-line to RL

Elicit the reasoning, reward the steps – then optimize with RL.

Where this feeds the reasoning-model wave



- The step-scored reward is exactly the **process supervision** in GRPO (**[LLM-1]**): spread the group-normalized reward over *steps*, not just the final token.
- **DeepSeek-R1** (**[LLM-11]**) and **Qwen3** (**[LLM-8]**) use RL to *elicit* long chains of thought – the very behavior Wei showed was latent.

CoT gives the reasoning to reward; the PRM says *which steps* deserve it; RL turns that signal into a better policy.

Recap

- **Chain-of-thought** prompting elicits reasoning from a frozen model – big gains on arithmetic/commonsense/symbolic tasks.
- The benefit is **emergent with scale**: small models get no help (or worse); large models jump (GSM8K 17.9 $\rightarrow$ 56.9).
- **Self-consistency**: sample many chains, majority-vote the answer (56.5 $\rightarrow$ 74.4), no training.
- **Reliability gap**: right answer via wrong steps – outcome-only rewards luck.
- **PRM** $>$ **ORM** $>$ majority on MATH (78.2 / 72.4 / 69.6); process supervision is more performant *and* more aligned; PRM800K released.

Next: this process signal is the reward RL optimizes – GRPO (**[LLM-1]**), and the reasoning models DeepSeek-R1 (**[LLM-11]**) / Qwen3 (**[LLM-8]**) build on it.